**Design note:** this workbook is meant to be read on screen while students code in VS Code. It is written to support prediction, experimentation, and modification in `scratch.py`, not passive reading.

# Part I — Foundations of Programming in Python

This workbook is the foundation for the rest of the course. The goal is not to memorize Python syntax. The goal is to understand how Python stores information, how Python changes information, and how to investigate behavior when something is unclear. This is a learning document, not a compact reference document.

**Recommended workflow:** predict what the code will do, run it, observe carefully, change one thing, and run it again.

## Contents


1. How to Use This Workbook
2. Computers Store Data and Change Data
3. Objects and Data Types
4. Working with Strings
5. Working with Lists
6. Working with Tuples
7. Working with Dictionaries
8. Variables, References, and Objects
9. Mutability, Immutability, Aliasing, and Copying
10. Expressions, Statements, and Operations
11. Collections and Data Representation
12. Boolean Logic, Comparisons, and Control Flow
13. Loops, Range, and Trace Tables
14. Nested Lists, 2-D, 3-D, and 4-D Structures
15. Functions, Methods, and Function Signatures
16. Importing Modules and Paths in VS Code
17. Files and JSON
18. Using AI the Right Way

19. Python Mental Model Summary
20. Integrated Problems and Transition to Part II: STI

# 1. How to Use This Workbook

Our first job is to build language fluency and a reliable mental model of how Python behaves. You are not just learning Python. You are learning how to investigate a system.

Whenever something feels unclear, do not stare at the code passively. Create a tiny exercise and run it. Small exercises are one of the fastest ways to turn confusion into understanding.

A good exercise is usually only three to ten lines long.

### Recommended working habit

Create a file called `scratch.py` in VS Code. Use it whenever you want to test a small idea without touching your main program.

```
a = [1, 2]
b = a
b.append(3)

print(a)
print(b)
```

Then change one thing and rerun. For example, replace `b.append(3)` with `b = b + [3]`. Does the behavior stay the same?

### Section 1 Questions

1. Why is a small test file like `scratch.py` useful?
2. What is the advantage of changing only one thing at a time in an experiment?

### ▶ Exercise Panel — Use a Scratch File

When something in Python is unclear, open `scratch.py` in VS Code and test the smallest possible version of the idea.

### 🧠 Mental Model Panel

Learning to program is partly learning to ask better questions. Small exercises produce evidence.

### 💡 Environment Reminder

In this course, students open the folder `student_repo` in VS Code. That folder lives under `C:\PythonClass`.

# 2. Computers Store Data and Change Data

Nearly every program can be understood using two questions:

- What information is stored?
- What operations change that information?

A spreadsheet stores numbers and updates them when formulas run. A game stores positions and scores and updates them when the player acts. A banking program stores balances and changes them when transactions occur. A portfolio program stores clients, positions, and cash, then changes them when a trade is recorded.

```
balance = 100
print(balance)
balance = balance + 25
print(balance)
```

Before running the code, predict the output. Then run it. Then change the starting value and rerun.

### Questions worth asking

- What value is stored before the update?
- What value is stored after the update?
- Which line changes the stored value?
- What value is produced by `balance + 25`?
- Would the same pattern hold if we updated the value twice?

### Section 2 Questions

1. In the example above, which line performs the update?
2. What is the difference between the value stored before the update and the value produced by `balance + 25`?

### 💡 Concept Panel

The central idea of this section is simple: computers store data and change data.

### ⚠ Warning Panel

When you read code, do not treat all lines as doing the same thing. Ask:

- Is this line computing a value?
- Is this line creating or rebinding a reference?
- Is this line changing an existing object?
- Is this line controlling the flow of execution?

# 3. Objects and Data Types

In Python, every value you work with is an object. Numbers are objects. Strings are objects. Lists and dictionaries are objects.

```
object = data + operations
```

| Type | Example | What it models |
|---|---|---|
| `int` | `7` | Whole numbers |
| `float` | `3.14` | Decimal numbers |
| `str` | `"apple"` | Text |
| `bool` | `True` | True/False conditions |
| `list` | `[1, 2, 3]` | Ordered sequence |
| `tuple` | `(3, 5)` | Fixed ordered grouping |
| `set` | `{1, 2, 3}` | Unique values |
| `dict` | `{"AAPL": 10}` | Key/value mapping |

```
x = 7
y = 3.14
name = "Alice"
flag = True
items = [1, 2, 3]

print(type(x))
print(type(y))
print(type(name))
print(type(flag))
print(type(items))
```

### 💡 What is a String?

A string is text made from characters you can type on a keyboard.

```
"hello"
"AAPL"
"123 Main Street"
```

### 💡 What is a List?

A list is an ordered sequence, like a shopping list.

```
["milk", "bread",
"eggs"]
```

### 💡 What is a Dictionary?

A dictionary stores keys and values. A **key** is the label used to look up a value.

```
{"AAPL": 10, "MSFT":
5}
```

### 🧠 Number vs Numeral

`7` is a number. `"7"` is text that represents a number.

```
7 + 7        # 14
"7" + "7"    # "77"
```

**Section 3 Questions**

1. What is the difference between a number and a numeral?
2. What kind of data structure would you choose for fast lookup by name?
3. Explain what the phrase `object = data + operations` means in your own words.

## 🧩 Symbols Matter

`[]` is used for lists, indexing, and slicing.
`{}` is used for dictionaries (and sets).
`()` is used for grouping, tuples, and function calls.

# 4. Working with Strings

A string is a sequence of characters. Students need a basic toolkit for manipulating strings because strings appear everywhere: names, menu choices, file paths, stock tickers, dates, and reports.

```
text = "Babson College"
```

### Indexing and slicing

The general slicing pattern is:

```
text[start:stop:step]
```

**start** is where the slice begins, **stop** is where it ends (not included), and **step** is how many positions to move each time.

```
print(text[0])      # first character
print(text[2:8])    # from index 2 up to but not
including 8
print(text[2:])     # from index 2 to the end
print(text[:5])     # from the beginning up to but not
including 5
print(text[::2])    # every second character
print(text[-1])     # last character
print(text[-3:])    # last 3 characters
```

Negative values count backward from the end of the string. So `-1` means “last position,” `-2` means “second-to-last,” and so on.

```
text = "Babson"

index:    0   1   2   3   4   5
char :    B   a   b   s   o   n

neg idx: -6  -5  -4  -3  -2  -1
```

### Most used string methods

### 💡 String Toolkit

The goal is not to memorize every method. The goal is to know the common ones you will actually use.

### ▶ Exercise Panel

```
text = " alice "
print(text.strip())
print(text.upper())
print(text.replace("a
", "A"))
```

Predict before running.

### ▶ Exercise Panel — Play with f-Strings

Copy the table example into VS Code and change the alignment, width, fill, and precision options. What changes visually?

| Method | Example | Result |
|---|---|---|
| `.upper()` | `"hello".upper()` | `"HELLO"` |
| `.lower()` | `"HELLO".lower()` | `"hello"` |
| `.strip()` | `" hi ".strip()` | `"hi"` |
| `.split(",")` | `"a,b,c".split(",")` | `['a','b','c']` |
| `.replace()` | `"cat".replace("c","b")` | `"bat"` |
| `.find()` | `"apple".find("p")` | `1` |
| `.join(...)` | `",".join(["a","b","c"])` | `"a,b,c"` |

`join` matters because many programs build text from pieces.

```
words = ["Computational", "Thinking", "with", "Python"]
sentence = " ".join(words)
print(sentence)
```

### f-Strings

This workbook teaches a small, practical subset of the part that comes after the colon `:`.

```
f"{value:[fill][align][width][.precision][type]}"
```

### The 4 slots (in order)

1. **Fill** + **Align** (optional)
2. **Width** (optional; minimum field width)
3. **.Precision** (optional)
4. **Type** (optional)

### Align

- `<` left
- `>` right
- `^` center

### Fill

Any single character, but it only works when align is present. Examples: `0>`, `.^`, `-<`.

### Width

A number like `10` meaning minimum width.

### Precision

`.2` is common with floats, so `.2f` means 2 digits after the decimal.

### Type (common)

- `d` integer
- `f` fixed-point float
- `%` percent

### Golden examples

```
n = 42
x = 12.5
p = 0.3789
name = "Alice"

print(f"{n:>5}")       # right-align, width 5
print(f"{n:0>5}")      # zero-fill, right-align, width 5
print(f"{x:.2f}")      # 2 decimals
print(f"{x:8.2f}")     # width 8, 2 decimals
print(f"{p:.1%}")      # percent, 1 decimal
print(f"{name:^10}")   # center, width 10
```

### Using f-strings to build a small table

```
ticker = "AAPL"
shares = 10
price = 205.4

print(f"{'Ticker':^10} {'Shares':^10} {'Price':^10}")
print(f"{ticker:^10} {shares:^10d} {price:>10.2f}")
```

### Section 4 Questions

1. What is the difference between indexing and slicing?
2. Use `.join()` to convert a list of words into one sentence.
3. Write an f-string that prints a price with 2 decimal places.

# 5. Working with Lists

A list is an ordered sequence of objects. Lists are mutable, so they are one of the most important data structures in beginner Python.

```
numbers = [3, 7, 2]
```

```
mixed = [10, "AAPL", True]
nested = [[1, 2], [3, 4]]
records = [{"ticker": "AAPL", "shares": 10}, {"ticker":
"MSFT", "shares": 5}]
```

| Method | Example | What it does |
|---|---|---|
| `.append(x)` | `a.append(5)` | Add one item to the end |
| `.extend([...])` | `a.extend([4,5])` | Add multiple items |
| `.insert(i,x)` | `a.insert(1,9)` | Insert at a position |
| `.remove(x)` | `a.remove(3)` | Remove by value |
| `.pop()` | `a.pop()` | Remove and return last item |
| `.sort()` | `a.sort()` | Sort in place |
| `.reverse()` | `a.reverse()` | Reverse in place |
| `.copy()` | `b = a.copy()` | Make a shallow copy |

**Section 5 Questions**

1. What is the difference between `.append()` and `.extend()`?
2. Create a copy of a list and show that mutating the copy does not change the original.
3. Write a list containing a number, a string, and another list.

### ⚠ Important

Many list methods mutate the list. That means they change the existing object.

### ▶ Exercise Panel

```
items = [2, 4, 6]
items.append(8)
items.pop()
print(items)
```

### 🧠 Side Panel

`a.append(3)` mutates. `a = a + [3]` creates a new list and rebinds the variable.

# 6. Working with Tuples

A tuple is an ordered grouping that is typically not changed.

```
point = (3, 5)
date = (2026, 1, 20)
location = (42.3, -71.2)
```

Tuples support indexing and iteration, but not list-style mutation methods like `append()`.

Tuples are useful for communication and defensive programming. They tell the reader that the values represent one fixed concept.

**Section 6 Questions**

1. Why might a tuple be a better choice than a list for a date or location?
2. What happens if you try to call `append()` on a tuple?

### 💡 Side Panel

Use a tuple when you want to treat a grouping as one fixed package.

### ▶ Exercise Panel

Try indexing a tuple and a list. Then try calling `append()` on both. What happens?

# 7. Working with Dictionaries

A dictionary maps keys to values. A key is the label used to look up the corresponding value.

```
prices = {"AAPL": 205, "MSFT": 417}
```

### Access and update

```
print(prices["AAPL"])
prices["GOOG"] = 180
```

### Three common interfaces in beginner Python

```
prices["AAPL"]    # key lookup
text.upper()      # method call
len(text)         # function call
```

| Method | Example | What it does |
|---|---|---|
| `.keys()` | `d.keys()` | View all keys |
| `.values()` | `d.values()` | View all values |
| `.items()` | `d.items()` | View key/value pairs |
| `.get(k)` | `d.get("AAPL")` | Safe lookup |
| `.update({...})` | `d.update({"TSLA":3})` | Add or update entries |
| `.pop(k)` | `d.pop("AAPL")` | Remove and return a value |

### ▶ Exercise Panel — Run and Describe

Copy each example into `scratch.py`. Run it. Then describe what each one does.

```
portfolio = {"AAPL": 10, "MSFT": 5}

print(portfolio["AAPL"])
print(portfolio.keys())
print(portfolio.values())
print(portfolio.items())

for key in portfolio.keys():
    print(key)

for value in portfolio.values():
    print(value)

for key, value in portfolio.items():
    print(key, value)
```

### ⚠ Side Panel

`d["missing"]` can fail. `d.get("missing")` is often safer.

```
portfolio = {"AAPL": 10, "MSFT": 5}

for key in portfolio.keys():
    print(key)

for value in portfolio.values():
    print(value)

for key, value in portfolio.items():
    print(key, value)

names = ["Alice", "Bob", "Cara"]
scores = [88, 91, 79]

for i, name in enumerate(names):
    print(i, name)

for name, score in zip(names, scores):
    print(name, score)
```

### Section 7 Questions

1. What is a key?
2. What is the difference between `[key]`, dot notation, and a function call?
3. Loop through a dictionary using `.keys()`, `.values()`, and `.items()`.

# 8. Variables, References, and Objects

A variable is not an object. A variable is a label that refers to an object. It is better to think of a variable as a label than as a box.

```
variable name  →  reference  →  object
```

```
a = [1, 2]
b = a

print(id(a))
print(id(b))
```

```
a ─┐
   ├──► [1, 2]
b ─┘
```

This is aliasing. Two variables are different names for the same object.

**Section 8 Questions**

1. What does it mean for two variables to be aliases?
2. What does `id()` help you reason about?

### 🧠 Python Manages Memory

`id()` helps you reason about object identity. Python handles the underlying memory details.

# 9. Mutability, Immutability, Aliasing, and Copying

This is one of the most important conceptual sections in the workbook.

Some objects can change after they are created. Lists and dictionaries are common mutable objects.

Integers, floats, strings, and tuples behave as immutable objects in our examples.

When the value of an immutable object appears to change, Python creates a new object and the variable starts referring to that new object.

```
a = [1, 2]
b = a
b.append(3)
print(a)
print(b)
```

```
x = 10
y = x
x = 20
print(x)
print(y)
```

```
a = [1, 2, 3]
b = a.copy()
b.append(4)
print(a)
print(b)
```

| Action | What happens |
|---|---|
| Mutation | The existing object changes |
| Rebinding | The variable starts referring to a different object |
| Aliasing | Two variables refer to the same object |
| Copying | A new object is created with similar contents |

### ⚠ Common Mistake

`b = a` does not create a second list. It creates a second reference.

### ▶ Exercise Panel

Replace `b.append(3)` with `b = b + [3]`. Then print `a` and `b`. Explain the difference.

**Section 9 Questions**

1. What is the difference between mutation and rebinding?
2. Why does `b = a` not create a second list?
3. Why are variables labels rather than objects?

# 10. Expressions, Statements, and Operations

These distinctions matter because they help you debug and reason more precisely about what code is doing.

| Category | Meaning | Examples |
|---|---|---|
| Expression | Produces a value | `3 + 5`, `len(items)` |
| Operation | Computation used inside an expression | `+`, `*`, function call |
| Statement | Performs an action in the program | `x = 5`, `print(x)` |

```
x = len(items) + 3
```

Here, `len(items) + 3` is the expression. The function call and the `+` are operations inside that expression. The assignment is the statement.

**Section 10 Questions**

1. In `x = len(items) + 3`, which part is the statement?
2. Which parts are operations?

🧠 **Side Panel**

Operations are generally part of expressions. Statements organize actions in the program.

# 11. Collections and Data Representation

Different representations make different operations easier. This is one of the deepest ideas in programming.

```
positions_a = [("AAPL", 10), ("MSFT", 5)]
positions_b = {"AAPL": 10, "MSFT": 5}
```

```
portfolio = {
    "client": "Alice",
    "cash": 12000,
    "positions": [
        {"ticker": "AAPL", "shares": 10},
        {"ticker": "MSFT", "shares": 5}
    ],
    "notes": ["retirement", "taxable account"]
}
```

| From | To | Why? |
|---|---|---|
| Parallel lists | Dictionary | Fast lookup by key |
| Dictionary | List of pairs | Easy iteration / sorting |
| List of lists | Flat list | Simpler processing |
| Raw records | Grouped dictionary | Easier reporting by category |

**Section 11 Questions**

1. Why might a dictionary be better than two parallel lists?
2. Convert a dictionary into a list of pairs.
3. Explain what it means to “reshape data.”

### 🧩 Design Panel

Ask: what representation makes the next operation easier?

# 12. Boolean Logic, Comparisons, and Control Flow

Boolean values represent True or False. They are the basis for decisions in a program.

### Comparison operators

| Operator | Meaning |
|---|---|
| `==` | equal to |
| `!=` | not equal to |
| `<` | less than |
| `>` | greater than |
| `<=` | less than or equal to |
| `>=` | greater than or equal to |

### Logical operators

```
a and b
a or b
not a
x in collection
```

Be careful not to confuse `not x` with `x != y`.

```
not is_member      # negate a truth value
x != y             # compare two values for inequality
```

### Arithmetic operators

```
+   -   *   /   //   %   **
```

Evaluation is usually left to right, but ** behaves differently. When precedence matters, use parentheses.

### ▶ Exercise Panel — Circle a List with %

```
colors = ["red",
"green", "blue"]

for i in range(8):
    print(colors[i %
len(colors)])
```

Why does the list repeat instead of failing?

### ⚠ Side Panel

Do not memorize large precedence tables. Use parentheses when clarity matters.

```
3 + 4 * 5
(3 + 4) * 5
2 ** 3 ** 2
```

### Selection with if / elif / else

```
score = 75

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("Below C")
```

Python checks conditions from top to bottom and stops once one branch succeeds.

### De Morgan's Laws

These are two very useful rewrite rules for boolean expressions:

```
not (A and B) == (not A) or (not B)
not (A or B)  == (not A) and (not B)
```

Truth tables are an important tool for checking whether two logical expressions are equivalent.

### Truth table for `not (A and B)`

| A | B | A and B | not (A and B) | not A | not B | (not A) or (not B) |
|---|---|---|---|---|---|---|
| True | True | True | False | False | False | False |
| True | False | False | True | False | True | True |
| False | True | False | True | True | False | True |
| False | False | False | True | True | True | True |

### Truth table for `not (A or B)`

| A | B | A or B | not (A or B) | not A | not B | (not A) and (not B) |
|---|---|---|---|---|---|---|
| True | True | True | False | False | False | False |
| True | False | True | False | False | True | False |
| False | True | True | False | True | False | False |
| False | False | False | True | True | True | True |

### Check the same laws in Python

```
cases = [(True, True), (True, False), (False, True),
(False, False)]

for a, b in cases:
    left = not (a and b)
    right = (not a) or (not b)
    print(a, b, left, right)

print()

for a, b in cases:
    left = not (a or b)
    right = (not a) and (not b)
    print(a, b, left, right)
```

### Section 12 Questions

1. Contrast `not x` with `x != y`.
2. Why does order matter in an `if / elif / else` chain?
3. Run the De Morgan examples. Why do the last two columns match?

# 13. Loops, Range, and Trace Tables

Loops repeat work. They are where programs begin to feel dynamic over time.

### for loops

```
numbers = [2, 4, 6]
for x in numbers:
    print("Number:", x)
    print("Double:", x * 2)
```

The loop variable `x` takes on one value at a time from the list.

### range

```
range(stop)
range(start, stop)
range(start, stop, step)
```

```
for i in range(5):
    print(i)

for i in range(2, 8):
    print(i)

for i in range(2, 10, 2):
    print(i)

for i in range(10, 0, -2):
    print(i)
```

### Accumulation

```
numbers = [2, 4, 6]
total = 0
for x in numbers:
    total = total + x
print(total)
```

### ▶ Exercise Panel — Run and Describe

Run each example and describe what each highlighted statement does.

```
for x in [1, 2, 3,
4]:
    if x == 3:
        break
    print(x)

for x in [1, 2, 3,
4]:
    if x == 3:
        continue
    print(x)

if True:
    pass
```

### 🧠 Side Panel — Why Trace?

Tracing makes program behavior visible. It is especially useful when loops feel confusing.

| Step | x | total |
|---|---|---|
| start | — | 0 |
| after iteration 1 | 2 | 2 |
| after iteration 2 | 4 | 6 |
| after iteration 3 | 6 | 12 |

### while loops

```
count = 0
while count < 3:
    print(count)
    count = count + 1
```

### `break`, `continue`, and `pass`

```
for x in [1, 2, 3, 4]:
    if x == 3:
        break
    print(x)
```

```
for x in [1, 2, 3, 4]:
    if x == 3:
        continue
    print(x)
```

```
if True:
    pass
```

`break` exits the loop. `continue` skips to the next iteration. `pass` is a placeholder that does nothing.

### Section 13 Questions

1. Use `range(start, stop, step)` to print even numbers from 2 to 10.
2. What is the purpose of an accumulator variable like `total`?
3. What is the difference between `break` and `continue`?

# 14. Nested Lists, 2-D, 3-D, and 4-D Structures

A 2-D list is usually best thought of as a **list of lists**.

### Build a 2-D list with nested loops

```
board = []

for row in range(2):
    current_row = []

    for col in range(3):
        current_row.append(0)

    board.append(current_row)

print(board)
```

```
board[row][column]

board[0]     → [0, 0, 0]
board[1]     → [0, 0, 0]
board[0][2]  → 0
```

### Build a 3-D list with nested loops

```
cube = []

for layer in range(2):
    board = []

    for row in range(2):
        current_row = []

        for col in range(2):
            current_row.append(0)

        board.append(current_row)

    cube.append(board)

print(cube)
```

### ⚠ Trap Exercise

Run the `bad_board` example, mutate one cell, and explain why more than one row changes.

### 💡 Side Panel — 1-D vs 2-D

A single loop often corresponds to a 1-D sequence. A nested loop often corresponds to a 2-D structure.

```
x = [
    [["a", "b"], ["c", "d"]],
    [[1, 2], [3, 4]]
]
```

```
x[0]       → [["a", "b"], ["c", "d"]]
x[0][0]    → ["a", "b"]
x[0][0][0] → "a"
```

### Build a 4-D list with nested loops

```
hyper = []

for block in range(2):
    cube = []

    for layer in range(2):
        board = []

        for row in range(2):
            current_row = []

            for col in range(2):
                current_row.append(0)

            board.append(current_row)

        cube.append(board)

    hyper.append(cube)

print(hyper)
```

### A dangerous pattern

```
bad_board = [[0] * 3] * 2
bad_board[0][0] = 99
print(bad_board)
```

This creates repeated references to the same inner list.

```
good_board = []

for row in range(2):
    current_row = []

    for col in range(3):
        current_row.append(0)

    good_board.append(current_row)

print(good_board)
```

### Section 14 Questions

1. What does `x[0]` mean in the 3-D example? What about `x[0][0]` and `x[0][0][0]`?
2. Why is `bad_board` dangerous?
3. Create a 3-D list with nested loops and explain what each dimension means.

# 15. Functions, Methods, and Function Signatures

A function packages a computation behind a name. The goal is not just shorter code. The goal is clearer thinking, less repetition, and better decomposition.

### Why functions matter: a report writer example

```
# repeated code
ticker = "AAPL"
shares = 10
price = 205.4
value = shares * price
print(f"{ticker:^10} {shares:^10d} {price:>10.2f}
{value:>12.2f}")

ticker = "MSFT"
shares = 5
price = 417.2
value = shares * price
print(f"{ticker:^10} {shares:^10d} {price:>10.2f}
{value:>12.2f}")
```

```
def print_header():
    print(f"{'Ticker':^10} {'Shares':^10} {'Price':^10}
{'Value':^12}")
    print("-" * 46)

def print_row(ticker, shares, price):
    value = shares * price
    print(f"{ticker:^10} {shares:^10d} {price:>10.2f}
{value:>12.2f}")

def print_report(positions):
    print_header()

    for ticker, shares, price in positions:
        print_row(ticker, shares, price)

positions = [
    ("AAPL", 10, 205.4),
    ("MSFT", 5, 417.2),
    ("GOOG", 3, 152.8)
]

print_report(positions)
```

### 🧠 Visual Model

```
definition → parameters
call       → arguments
```

### 💡 Functions Can Call Functions

In the report example, `print_report()` calls `print_header()` and `print_row()`. This is one way programmers decompose larger tasks.

### ▶ Exercise Panel

Copy the report-writer code into VS Code. Run it. Then work through the example line by line and trace which function calls which.

```
def add(a, b):
    return a + b

print(add(3, 4))
```

- **Function signature:** `add(a, b)`
- **Parameters:** names in the definition
- **Arguments:** values passed in the call
- **Return value:** the value sent back to the caller

```
def square(x):
    return x * x

def show_square(x):
    print(x * x)
```

```
len(items)       # function call
items.append(3)  # method call
```

### Section 15 Questions

1. Why is the report writer example better with functions than with repeated print statements?
2. What is the difference between a parameter and an argument?
3. What is the difference between `return` and `print`?
4. Did `show_square` return anything?

# 16. Importing Modules and Paths in VS Code

Python programs can import code written by others. This lets you reuse existing functionality instead of writing everything from scratch.

```
import math
print(math.sqrt(16))

from math import sqrt
print(sqrt(16))
```

### Project structure in this course

Students open `student_repo` in VS Code. That folder is located under `C:\PythonClass`.

```
C:\PythonClass
└── student_repo
    ├── src
    │   └── main.py
    ├── lib
    │   └── utils.py
    ├── data
    │   └── prices.csv
    └── sandbox
        └── experiments.py
```

### Current working directory (cwd)

The current working directory is the folder Python treats as “where I am right now.” Relative paths are interpreted from there.

```
import os
print(os.getcwd())
```

### Absolute vs relative paths

### 💡 Course Context

For this course, the current working directory is almost always `student_repo` or something inside it.

### 💡 Moving Up Folders

You can chain `..\..` to move up more than one level in a path.

### ▶ Exercise Panel — Sandbox Path Tests

Inside `student_repo\sandbox`, create a few folders and files. Then test paths from one file to another.

```
student_repo
└── sandbox
    ├── test_a
    │   └── hello.py
    └── test_b
        └── run_test.py
```

Then experiment with imports and paths using both absolute and relative styles.

| Type | Example | Meaning |
|---|---|---|
| Absolute path | `C:\PythonClass\student_repo\data\prices.csv` | Full path from the drive root |
| Relative path | `data\prices.csv` | Path relative to the current working folder |
| Relative path | `.\data\prices.csv` | The current folder, then the data folder |
| Relative path | `..\other_repoile.txt` | Go up one folder, then into another folder |

- `.` means the current folder
- `..` means the parent folder
- `\` and `/` are path separators; on Windows you will commonly see `\`

In VS Code, once students open `student_repo` as the project root, they often do not need to write `.` explicitly. A relative path like `data\prices.csv` is usually enough.

### Section 16 Questions

1. What is the difference between an absolute path and a relative path?
2. What does `.` mean? What does `..` mean?
3. Why do students often not need to write `.` explicitly when working inside `student_repo`?

# 17. Files and JSON

Programs often need to save data between runs. Files let information persist after the program stops.

### Reading text files

```
with open("data.txt", "r") as f:
    print(f.read())

with open("data.txt", "r") as f:
    print(f.readline())
    print(f.readline())

with open("data.txt", "r") as f:
    lines = f.readlines()
    print(lines)
```

### Writing and appending

When writing multiple lines, remember to add
where needed.

```
with open("notes.txt", "w") as f:
    f.write("Hello\\n")
    f.write("World\\n")

with open("notes.txt", "a") as f:
    f.write("Another line\\n")
```

### File position and `seek()`

```
with open("data.txt", "r") as f:
    print(f.read(5))
    f.seek(0)
    print(f.read(5))
```

In beginner text-file work, `seek()` is not common. It is more commonly discussed with binary files.

### What is a binary file?

A text file stores characters. A binary file stores raw bytes. Images, audio files, executable files, and many custom file formats are binary files.

### First pattern: CSV

### ▶ Exercise Panel — Run and Describe

Copy each file example into `scratch.py`. Run it. Then describe what each one does.

```
with open("data.txt",
"r") as f:
    print(f.read())

with open("data.txt",
"r") as f:

print(f.readline())

with open("data.txt",
"r") as f:

print(f.readlines())
```

### 💡 Why JSON Matters

JSON gives us a standard, readable way to store structured objects between runs.

**a) Create the object in Python**

```
positions = [
    ("AAPL", 10),
    ("MSFT", 5)
]

print(positions)
```

**b) Write it out to a CSV file**

```
positions = [
    ("AAPL", 10),
    ("MSFT", 5)
]

with open("positions.csv", "w") as f:
    f.write("ticker,shares\\n")
    for ticker, shares in positions:
        f.write(f"{ticker},{shares}\\n")
```

**c) Read it in from CSV**

```
with open("positions.csv", "r") as f:
    print(f.read())
```

**d) Recreate the object**

```
positions = []

with open("positions.csv", "r") as f:
    next(f)  # skip header

    for line in f:
        ticker, shares = line.strip().split(",")
        positions.append((ticker, int(shares)))

print(positions)
```

### Second pattern: JSON

**a) Create the object in Python**

```
positions = [
    {"ticker": "AAPL", "shares": 10},
    {"ticker": "MSFT", "shares": 5}
]

print(positions)
```

**b) Write it out to a JSON file**

```
import json

positions = [
    {"ticker": "AAPL", "shares": 10},
    {"ticker": "MSFT", "shares": 5}
]

with open("positions.json", "w") as f:
    json.dump(positions, f, indent=2)
```

**c) Read it in from JSON**

```
with open("positions.json", "r") as f:
    print(f.read())
```

**d) Recreate the object**

```
import json

with open("positions.json", "r") as f:
    positions = json.load(f)

print(positions)
```

With CSV-like text, you often have to split strings, convert types, and build the object manually. With JSON, the structure is already represented. That is one reason JSON makes life easier.

### Section 17 Questions

1. What is the difference between `read()`, `readline()`, and `readlines()`?
2. What is the difference between write mode and append mode?
3. Why does JSON make reconstruction of structured objects easier than CSV-like text?

# 18. Using AI the Right Way

AI can help learning, but it can also prevent learning if used poorly.

**Good uses**

- Ask for an explanation of an error message
- Ask for a simpler example of a concept
- Ask for hints or questions to think about
- Ask for a trace of code you do not understand
- Ask for feedback on your reasoning

**Bad uses**

- Submitting AI-generated code you do not understand
- Copying a full solution instead of reasoning through the problem
- Using AI to bypass the learning process

A good rule: if you cannot explain the code, you should not submit it.

**Section 18 Questions**

1. Give one good use of AI in this course.
2. Give one bad use of AI in this course.

**⚠ Warning Panel**

AI should support your thinking, not replace it.

# 19. Python Mental Model Summary

This section pulls together the main ideas from Part I into one place. Think of it as a compact reference page you can keep open while coding.

### Variable → Reference → Object

A variable is **not** an object. A variable is a label that refers to an object.

```
x = 10
y = x
x = 20

After rebinding:
x ──► 20
y ──► 10
```

### Mutation vs Rebinding

```
mutation  → object changes
rebinding → variable refers to a new object
```

```
numbers.append(4)        # mutation
numbers = numbers + [4]  # rebinding
```

### Objects contain data and operations

```
list
  data       → [1, 2, 3]
  operations → append(), pop(), sort()

string
  data       → "Babson"
  operations → upper(), split()
```

### Three common interfaces in Python

```
object[key]
object.method()
function(object)
```

### 🧠 Debugging Strategy

1. Simplify the program.
2. Print intermediate values.
3. Check types with `type()`.
4. Test small pieces in `scratch.py`.
5. Verify assumptions.

```
portfolio["AAPL"]   # key lookup
numbers.append(4)   # method call
len(numbers)        # function call
```

### Expressions vs Statements vs Operations

```
Expression → produces a value
Statement  → performs an action
Operation  → transforms data
```

### Core control-flow tools

```
if / elif / else → decisions
for              → iterate over elements
range()          → generate sequences
break            → exit loop
continue         → skip iteration
```

### Data structures you now know

| Structure | Example | When used |
|---|---|---|
| list | `[1, 2, 3]` | ordered sequence |
| tuple | `(x, y)` | fixed grouped data |
| dictionary | `{"AAPL": 10}` | key → value lookup |
| string | `"text"` | text data |

### Programs manage state

```
state       → stored data
transitions → code that changes state
invariants  → conditions that must remain true
```

### Section 19 Questions

1. What is the difference between mutation and rebinding?
2. What are the three common interfaces for interacting with objects?
3. Why is it useful to think of a variable as a label rather than as an object?

# 20. Integrated Problems and Transition to Part II: STI

### Integrated Problems

#### Easy (7)

1. Predict the output of a simple assignment example.
2. Identify the type of `"7"`.
3. Explain what `len([10,20,30])` returns.
4. Write an f-string that prints a name and score.
5. Use `.get()` on a dictionary.
6. What does `range(4)` produce?
7. Why is `scratch.py` useful?

#### Medium (7)

8. Convert parallel lists into a dictionary.
9. Convert a dictionary into a list of pairs.
10. Explain what `text[2:5]` returns.
11. Fill in a trace table for an accumulator loop.
12. Read a JSON example and describe the structure.
13. Write a relative path from `student_repo` to a file in a subfolder.
14. Write a text file, then append one more line.

#### Hard but Fair (7)

15. Explain why `b = a` is not copying.
16. Design a good representation for portfolio holdings.
17. Trace a nested loop over a 2-D list.
18. Explain how a 2-D list models a board.
19. Compare `a.append(3)` and `a = a + [3]`.
20. Create a 3-D list and explain what each dimension means.
21. State one good and one bad use of AI in this course.

### Transition to Part II: State–Transition–Invariant (STI)

Programs often model real systems. To model a system well, we need a better vocabulary for describing how the system works.

#### 🧩 STI Bridge

**State**: what information describes the system?
**Transition**: what changes it?
**Invariant**: what must stay true?

| Idea | Meaning | Bank account example |
|---|---|---|
| State | Information describing the system | balance |
| Transitions | Operations that change the state | deposit, withdraw |
| Invariants | Properties that must remain true | balance should not become negative |

```
Example: Tic-Tac-Toe

State:
  board
  current player

Transitions:
  place mark
  switch player

Invariants:
  each square is X, O, or blank
  only one mark per square
```

Part I was about understanding how Python behaves. Part II will begin with abstraction, decomposition, pattern recognition, and algorithms, then gradually formalize system descriptions using **State, Transitions, and Invariants**.

**Closing thought:** By the end of this part, you should not expect to know all of Python. That is not the goal. The goal is to understand enough syntax, structure, and behavior to reason about programs and prepare for computational modeling.